

Friday: A Local Retrieval-Augmented Generation System for Privacy-Preserving PDF Intelligence

Shreya Sharma and Steve Uhlig

Abstract. Friday is a local Retrieval-Augmented Generation (RAG) system for searching and questioning private PDF collections without sending document data to a cloud service. The implementation combines a Flask API, SQLite, ChromaDB, PyMuPDF, the all-MiniLM-L6-v2 embedding model, and Ollama. This revised paper focuses its evaluation claims on evidence available from the supplied implementation. A repository audit using the production extraction and chunking logic identifies 706 PDFs, 1,397 pages, 4,165,134 extracted characters, and 6,283 non-empty chunks. The paper describes the implemented architecture and a reproducible evaluation methodology for the retrieval choices that matter in a personal knowledge-management setting: chunking, embedding model, hybrid retrieval, and HNSW configuration. Earlier pilot retrieval figures based on 80 manually constructed queries are treated as exploratory rather than as statistically general conclusions. The resulting contribution is an implementable privacy-preserving system and a defensible measurement framework for its further evaluation.

Keywords: retrieval-augmented generation; local AI; PDF retrieval; semantic search; ChromaDB; HNSW; reproducibility

1. Introduction

Document assistants frequently depend on cloud APIs, requiring sensitive notes, reports, and PDFs to leave the user's device. This is problematic when documents contain personal, commercial, legal, or regulated information. Friday addresses this problem with a locally hosted RAG application. A user creates named collections, indexes PDFs, searches them semantically, and can ask a local language model questions using retrieved passages as context.

The engineering goal is not merely to demonstrate that dense retrieval can outperform a lexical method on a small query set. For the intended use case, the important design questions are how documents are chunked, which local embedding model provides the best quality-cost trade-off, whether lexical and dense signals are complementary, and how approximate nearest-neighbour settings affect latency and retrieval quality. This paper therefore separates implementation evidence from claims requiring a fuller experiment.

2. Related Work

RAG conditions generation on retrieved non-parametric evidence, reducing reliance on a model's parametric memory [1]. Dense retrieval encodes queries and passages in a shared embedding space, allowing matches beyond exact term overlap [2]. ChromaDB provides an embedded vector-store option suitable for local applications, while HNSW supports approximate nearest-neighbour search [3]. BM25 remains a useful lexical diagnostic [4], but it is not the only or primary comparator for a personal-document retrieval system. Evaluation should also examine dense-model choices, chunk boundaries, hybrid ranking, and ANN approximation relative to exact dense search.

2.1 Research Questions and Contributions

This study addresses three questions: RQ1: how can a private PDF collection be indexed and queried using entirely local components? RQ2: which retrieval design choices should be evaluated for a local knowledge-management use case? RQ3: what evidence is required before making claims about retrieval quality and system performance? The paper contributes (i) a concrete local RAG implementation; (ii) a code-derived audit of the available evaluation corpus; and (iii) a reproducible protocol that aligns retrieval and indexing evaluation with the design decisions made by the system.

This scope intentionally distinguishes system construction from system validation. A functional prototype demonstrates that the architecture can be built and operated locally. Claims about comparative quality, scalability, and answer faithfulness need a controlled workload, recorded ground truth, and repeated measurement. Treating these as separate contributions avoids turning incomplete experiments into stronger claims than the evidence supports.

3. System Design and Implementation

Friday follows a browser-to-local-server architecture. The frontend is implemented in HTML, CSS and JavaScript. A Flask backend supplies authenticated REST endpoints, a streaming Server-Sent Events chat endpoint, PDF indexing, and search. SQLite stores users, sessions, named groups, and indexed-file metadata. Each group maps to an isolated persistent ChromaDB collection. Ollama runs a locally installed generative model, while all-MiniLM-L6-v2 creates 384-dimensional text embeddings.

Component	Implementation role
PyMuPDF	Extracts text and page metadata from PDF files.
Chunking	Sliding windows of 800 characters with 100-character overlap.
SentenceTransformer	Encodes each chunk and query with all-MiniLM-L6-v2.
ChromaDB	Stores chunk vectors and runs cosine HNSW search.
SQLite	Persists users, sessions, groups, and file provenance.
Ollama + SSE	Generates local context-grounded answers and streams tokens.

3.1 Indexing pipeline

The indexing route extracts text with PyMuPDF, calculates an MD5 file hash for duplicate detection, splits extracted text into overlapping fixed-size character chunks, encodes chunks in batches, and upserts vectors into the selected ChromaDB collection. Metadata records filename, path, hash, and chunk index. The current code reports progress using newline-delimited JSON. These implementation details are important because they determine what is measured as indexing: extraction, chunking, embedding, and vector persistence are distinct costs.

3.2 Retrieval and generation

At query time, Friday embeds the question with the same model and requests the top k cosine-nearest chunks from selected collections. The application default is k = 6. Retrieved text is assembled into a context block before the request is sent to a local Ollama model. The prompt asks the model to answer from supplied context and to state when the answer is absent. This is a mitigation strategy, not a guarantee of factuality.

3.3 Security and scope

The code implements session-based authentication, named collection isolation, and local persistence. The supplied code uses SHA-256 password hashes; for a deployment-facing system this should be replaced by a salted adaptive password hash such as Argon2 or bcrypt. Likewise, “local” means the configured inference, embedding, and database operations run on the device; actual network-egress claims should be verified with a recorded network-monitoring procedure.

4. Corpus Audit

A complete audit was run over the repository’s data directory using the same PyMuPDF extraction method and 800-character/100-character-overlap sliding window used in backend/app.py. The audit processed 706 PDF files across eight domain directories. It yielded 1,397 pages, 4,165,134 extracted characters, and 6,283 non-empty chunks. Under the current implementation, this corresponds to 6,283 vectors before any duplicate or extraction-error exclusions.

Domain	PDFs	Pages	Chunks
Accountant	118	241	1057
Advocate	118	244	1104
Agriculture	63	132	593
Apparel	97	188	821
Arts	103	199	880
Automobile	36	72	322
Aviation	117	219	1041
Banking	54	102	465
Total	706	1397	6283

5. Evaluation Methodology

The corpus audit is a reproducible content inventory, not a retrieval-quality result. Retrieval quality requires queries with documented relevance judgements. The original 80 manually constructed queries may remain as a pilot/development set, but they are too limited and insufficiently documented to support strong general claims or statistical comparison. The final evaluation should use a frozen versioned query file with at least 15-20 realistic information needs per domain, a separation between development and test queries, and target document/chunk identifiers.

5.1 Relevance assessment and uncertainty

Each retrieved chunk should be labelled relevant when it contains sufficient evidence to answer its corresponding question. Two assessors should label a subset independently; agreement should be reported with Cohen's kappa, and adjudication rules should be recorded. Report Recall@k, MRR, and nDCG@k per domain as well as overall. For any paired comparison, calculate a 95% stratified bootstrap confidence interval over per-query metric differences, resampling within domains. A comparison should not be presented as an improvement when its interval includes zero.

5.2 Retrieval design study

Condition	Controlled comparison	Report
Embedding	all-MiniLM-L6-v2; bge-small-en-v1.5; all-mpnet-base-v2	Recall@6, MRR, nDCG@6, ms/chunk, vector storage.
Chunking	Fixed 400/800/1200 characters; paragraph-aware chunks	Quality metrics, chunk count, index size, end-to-end indexing time.
Hybrid retrieval	Dense only; BM25 only; reciprocal-rank fusion	Quality and p50/p95 retrieval latency.
HNSW	Default HNSW settings compared with exact flat dense search	Neighbour agreement, relevant retrieval, p50/p95 latency, index size.

BM25 is retained as a lexical diagnostic. It tests whether exact term matching covers some queries that dense retrieval misses; it is not sufficient evidence on its own for Friday's core design. The exact dense baseline has a separate role: it measures the retrieval cost of approximate HNSW search while keeping embeddings fixed.

5.3 Indexing and systems measurement

Indexing throughput must be measured by content processed, not documents processed. Each run should log accepted PDFs, pages, extracted characters, chunks/vectors written, extraction time, embedding time, database-upsert time, total wall-clock time, peak memory, and index size. Run one cold trial and at least five warm trials; report the median, interquartile range, and raw run values. Machine model, CPU/GPU, RAM, storage, operating system, Python, ChromaDB, and model versions are required for reproducibility.

For query performance, report p50 and p95 query-embedding, vector-search, and end-to-end retrieval latency. For generated answers, report time to first token and tokens per second separately. Corpus growth must be expressed in vector/chunk count (for example, 500, 2,500, 5,000 and 7,500 vectors), not documents. Single-machine observations are configuration-specific and cannot establish general system performance.

6. Results Status and Discussion

Claim or result	Status and appropriate interpretation
Pilot dense vs BM25 figures from 80 queries	Exploratory point estimates only. They may motivate further testing but do not demonstrate robust superiority without a frozen set, relevance protocol, and uncertainty analysis.
Flat dense, hybrid, chunking, embedding, LLM, and HNSW ablations	Not measured in the supplied materials. They are evaluation conditions, not results; no numeric values are reported here.
Latency, token rate, and indexing throughput	Any prior figures are single-machine observations without enough method detail to be reproducible. Future reports must follow the workload protocol in Section 5.3.
Out-of-context refusal rate	Not reported as a final result because supporting prompt, output, and adjudication logs are not available. Context prompting is described as a mitigation strategy only.

The revised evidence supports an engineering conclusion: Friday is a working local PDF intelligence system and its repository contains a substantial heterogeneous corpus for controlled experiments. It does not yet support broad claims of retrieval superiority, general throughput, or a precise hallucination-refusal rate. This distinction strengthens the paper because it makes its contribution testable rather than overstated.

The primary design trade-offs are visible in the implementation. Smaller chunks may improve localisation but fragment context; larger chunks reduce vector count but can dilute relevance and overfill the generation prompt. Larger embedding models may improve semantic matching but increase local CPU cost and storage. HNSW can reduce search latency relative to exhaustive dense search, but its approximation must be measured. Hybrid retrieval may improve named-entity and numerical queries, which are common in personal reports and technical PDFs.

7. Conclusion

Friday implements a local RAG workflow for private PDF collections using Flask, SQLite, ChromaDB, PyMuPDF, SentenceTransformers, and Ollama. A code-derived repository audit confirms 706 PDFs, 1,397 pages, and 6,283 chunks under the production chunking configuration. The corrected evaluation framing treats earlier manual-query retrieval scores as a pilot and prioritises reproducible comparisons of embedding models, chunking policies, hybrid ranking, and HNSW approximation. This provides a credible basis for an MSc engineering contribution while identifying the empirical work needed to substantiate performance claims.

References

- [1] P. Lewis et al., 'Retrieval-augmented generation for knowledge-intensive NLP tasks,' NeurIPS, 2020.
- [2] N. Reimers and I. Gurevych, 'Sentence-BERT: Sentence embeddings using siamese BERT-networks,' EMNLP, 2019.
- [3] Y. A. Malkov and D. A. Yashunin, 'Efficient and robust approximate nearest neighbor search using HNSW,' IEEE TPAMI, 2020.
- [4] S. Robertson and H. Zaragoza, 'The probabilistic relevance framework: BM25 and beyond,' Foundations and Trends in Information Retrieval, 2009.
- [5] N. Thakur et al., 'BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models,' NeurIPS Datasets and Benchmarks, 2021.

Appendix A. Reproducibility Record

This paper was generated from the Friday repository. The corpus quantities in Section 4 were produced by a code audit that recursively reads the repository PDF data, extracts text with PyMuPDF, and applies the same fixed-character chunking window as the application. The audit artefact records per-file domain, page count, extracted character count, and chunk count. The document generator is retained with the PDF so the paper can be regenerated after the corpus changes.

To complete the empirical study, archive the following artefacts with the submission: (i) a versioned query and relevance-label file; (ii) source code/commit identifier; (iii) configuration file with chunking, model, and HNSW settings; (iv) per-run timing and resource logs; (v) raw retrieval ranks; (vi) bootstrap analysis output; and (vii) a clear account of assessor agreement and exclusions. These artefacts turn evaluation claims into reproducible evidence.

Appendix B. Reporting Checklist

Area	Minimum report
Corpus	PDF count, page count, extracted characters, chunks/vectors, exclusions, per-domain distribution.
Indexing	Cold/warm runs; extraction, embedding and upsert times; pages/chunks/vectors per minute; memory; index size.
Retrieval	Frozen queries; labels; assessor agreement; Recall@k, MRR, nDCG@k; per-domain scores; 95% confidence intervals.
System	Machine and software versions; p50/p95 latency; concurrency; warm-up; workload; raw results.
Claims	State which figures are pilot observations and which comparisons have statistical support.